

K Closest Points to Origin Given an array of points where $\text{points}[i] = [x_i, y_i]$ represents a point on the X-Y plane and an integer k , return the k closest points to origin $(0,0)$.

The distance between two points on the X-Y plane is the Euclidean distance $\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$

You may return the answer in any order. The answer is guaranteed to be unique.

Ex: $\text{points} = [[1,3], [-2,2]]$ $k=1$

$$\sqrt{(0-1)^2 + (0-3)^2} \Rightarrow \sqrt{1+9} = \sqrt{10}$$

$$\sqrt{(0-(-2))^2 + (0-2)^2} = \sqrt{4+4} = \sqrt{8}$$

$\sqrt{8} < \sqrt{10}$, so $(-2, 2)$ is closer to origin

Ex: $\text{points} = [[3,3], [5,-1], [-2,4]]$ $k=2$

$$\sqrt{(0-3)^2 + (0-3)^2} \Rightarrow \sqrt{9+9} = \sqrt{18}$$

$$\sqrt{(0-5)^2 + (0-(-1))^2} = \sqrt{25+1} = \sqrt{26}$$

$$\sqrt{(0-(-2))^2 + (0-4)^2} = \sqrt{4+16} = \sqrt{20}$$

$[3,3], [-2,4]$ are closer to origin

Brute force Approach

1. Use a custom comparator to sort the points by distance
2. Return the first k elements

TC: $O(n \log n)$ SC: $O(1)$

Optimized Approach

1. We only need k closest, not full sorting
2. Use a Max Heap of size k
3. Keep only k closest points seen so far by replacing in heap

```
public int[][] kClosest(int[][] points, int k) {
```

```
// Max heap based on distance
```

```
    PriorityQueue<int[]> maxHeap = new PriorityQueue<>(  
        (a, b) -> {
```

```
            int dist1 = a[0]*a[0] + a[1]*a[1];
```

```
            int dist2 = b[0]*b[0] + b[1]*b[1];
```

```
            return dist2 - dist1; // max heap
```

```
        }  
    );
```

```
    for (int[] point : points) {
```

```
        maxHeap.offer(point);
```

```
        // Keep heap size <= k
```

```
        if (maxHeap.size() > k) {
```

```
            maxHeap.poll();
```

```
        }
```

```
    }
```

```
// Extract result
```

```
int[][] result = new int[k][2];
```

```
int idx = 0;
```

```
while (!maxHeap.isEmpty()) {
```

```
    result[idx++] = maxHeap.poll();
```

```
}
```

```
return result;
```

```
}
```

TC: $O(n \log k)$

SC: $O(k)$ $\uparrow$ heap